

Generic List<T> Collection Class in C# with Examples

In this article, I am going to discuss the **Generic List<T> Collection Class in C#** with Examples. Please read our previous article where we discussed [How to implement Generics in C#](#) with Examples. The Generic List<T> Class in C# is a collection class that is present in **System.Collections.Generic** namespace. The List Collection class is one of the most widely used generic collection classes in real-time applications. At the end of this article, you will understand the following pointers with Examples.

1. [What is Generic List<T> Collection in C#?](#)
2. [How to Create a List in C#?](#)
3. [How to Add Elements into a Generic List<T> Collection in C#?](#)
4. [How to Access a Generic List<T> Collection in C#?](#)
5. [How to Insert Elements at a Specific Position in a C# List?](#)
6. [How to Check the Availability of an Element in a List Collection in C#?](#)
7. [How to Remove Elements from a Generic List Collection in C#?](#)
8. [How to Copy an Array to a List in C#?](#)
9. [Generic List Collection with Complex Type in C#](#)
10. [How to Find Element in a Generic List Collection in C#?](#)
11. [How to Sort a List of Simple Type and Complex Types in C#?](#)

What is Generic List<T> Collection in C#?

The Generic List<T> in C# is a Collection Class that belongs to System.Collections.Generic namespace. This Generic List<T> Collection Class represents a strongly typed list of objects which can be accessed by using the integer index which is starting from 0. It also provides lots of methods that can be used for searching, sorting, and manipulating the list of items.

We can create a collection of any data type by using the Generic List<T> Collection Class in C#. For example, if you want then you can create a list of strings, a list of integers, and a list of doubles, and it is also possible to create a list of user-defined complex types such as a list of customers, a list of products, a list of students, etc. The most important point that you need to keep in mind is the size of the collection grows automatically when we add items to the collection.

How to Create a Generic List Collection in C#?

If you want to add elements to the generic list collection then you need to use the following Add() and AddRange() methods. If you notice the Add and Add method expects the data of type T. And here, T is nothing but the parameter type that you need to specify while creating the instance of the Generic List Class.

1. **Add(T item):** The Add(T item) method is used to add an element to the end of the Generic List. Here, the parameter item specifies the object to be added to the end of the Generic List. The value can be null for a reference type.
2. **AddRange(IEnumerable<T> collection):** The AddRange(IEnumerable<T> collection) method is used to add the Elements of the specified collection to the end of the Generic List. The parameter collection specifies the collection whose elements should be added to the end of the Generic List. The collection itself cannot be null, but it can contain elements that are null if type T is a reference type.

For example, Adding Elements using Add Method of the List Class. As you can see in the below code while creating the Generic List collection instance, we are specifying the type parameter as a string and this allows

the collection to store elements of string type only. In this case, the element will be added at the end of the collection.

```
List<string> countries = new List<string>();  
countries.Add("India");  
countries.Add("Srilanka");
```

As we created the above countries collection by specifying the type parameter as a string, so we can store only string type values in this collection. Other than string, the compiler will throw compile time error. The following two statements will give compile time errors as they are not string-type values.

```
countries.Add(100);  
countries.Add(true);
```

Adding Elements using the AddRange Method of the List Class. In this case, we have two collections and we need to add one collection into another collection as follows. The following is our first collection.

```
List<string> countries = new List<string>();  
countries.Add("India");  
countries.Add("Srilanka");
```

The following is our second collection.

```
List<string> newCountries = new List<string>();  
newCountries.Add("USA");  
newCountries.Add("UK");
```

Now, we want to add our second collection at the end of the first collection. To do so, we need to use the AddRange method as follows:

```
countries.AddRange(newCountries);
```

Even, it is also possible in C# to create a List<T> collection using Collection Initializer Syntax as follows:

```
List<string> countries = new List<string>  
{  
    "India",  
    "Srilanka",  
    "USA"  
};
```

So, these are the different ways to add elements to the end of a generic list collection.

How to access a Generic List<T> Collection in C#?

We can access the elements of the Generic List<T> Collection in C# using three different ways. They are as follows:

Using Index to Access Generic List<T> Collection in C#:

The List<T> Class Implements the IList<T> interface. So, we can access the individual elements of the List Collection in C# by using the integral index. In this case, we just need to specify the index position of the element that we want to access. The Index is 0 Based. If the specified index is not present, then the compiler will throw an exception. The syntax is given below.

```
countries[0]; //First Element  
countries[1]; //Second Element  
countries[2]; //Third Element
```

Using For-Each Loop to Access Generic List<T> Collection in C#:

You can also use a for-each loop to access the elements of a Generic List<T> collection in C# as follows. Here, the loop variable must be of the same type as the type of elements you stored in the collection.

```
foreach (string item in countries)
{
    Console.WriteLine(item);
}
```

Even you can also use the var keyword to define the loop variable and based on the values it will automatically decide the data type as follows.

```
foreach (var item in countries)
{
    Console.WriteLine(item);
}
```

Using For Loop to Access Generic List<T> Collection in C#:

You can also access the Generic List<T> collection in C# using a for loop as follows. Here, we need to get the count of the list collection by using the Count properties of the List class and then start the loop from 0 and fetch the element of the list collection using the Index position. Here, the loop variable is going to be of integer which is going to point to the index position of the collection.

```
for (int i = 0; i < countries.Count; i++)
{
    var element = countries[i];
}
```

Example to Understand How to Create a List<T> Collection and Add Elements in C#:

For a better understanding of how to create a Generic List<T> Collection in C# and how to add elements to the list collection using Add and AddRange method, and how to access the elements of the generic list collection in C#, please have a look at the below example. The following code is self-explained, so please go through the comment line.

```
using System;
using System.Collections.Generic;
namespace GenericListCollectionDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Generic List of string type to store string elements
            List<string> countries = new List<string>();
            //Adding String Elements to the Generic List using the Add Method
            countries.Add("INDIA");
            countries.Add("USA");
            //The following Two Statements will give compile time error as ele
            string type
            //countries.Add(100);
            //countries.Add(true);
            //Creating Another Generic List Collection of String Type
            List<string> newCountries = new List<string>();
            //Adding Elements using Add Method
            newCountries.Add("JAPAN");
        }
    }
}
```

Method

```
newCountries.Add("UK");
//Adding the newCountries collection into countries collection using

countries.AddRange(newCountries);
//Accessing Generic List Elements using ForEach Loop
Console.WriteLine("Accessing Generic List using For Each Loop");
foreach (var item in countries)
{
    Console.WriteLine(item);
}
//Accessing Generic List Elements using For Loop
Console.WriteLine("\nAccessing Generic List using For Loop");
for (int i = 0; i < countries.Count; i++)
{
    var element = countries[i];
    Console.WriteLine(element);
}
//Accessing List Elements by using the Integral Index Position
Console.WriteLine("\nAccessing Individual List Element by Index Position");
Console.WriteLine($"First Element: {countries[0]}");
Console.WriteLine($"Second Element: {countries[1]}");
Console.WriteLine($"Third Element: {countries[2]}");
Console.WriteLine($"Fourth Element: {countries[3]}");
Console.ReadKey();
}
```

Output:

```
Accessing Generic List using For Each Loop
INDIA
USA
JAPAN
UK

Accessing Generic List using For Loop
INDIA
USA
JAPAN
UK

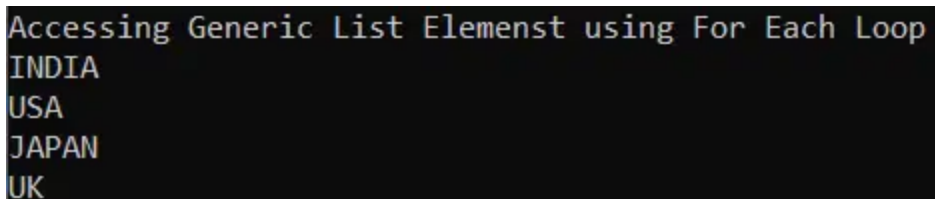
Accessing Individual List Element by Index Position
First Element: INDIA
Second Element: USA
Third Element: JAPAN
Fourth Element: UK
```

Example to Add Elements to a List using Collection Initializer in C#:

This is a new feature added to C# 3.0 which allows initializing a collection directly at the time of declaration like an array. In the below example, we are using Collection Initializer syntax instead of the Add method of the Generic List Collection class to add elements into the collection object in C#.

```
using System;
using System.Collections.Generic;
namespace GenericListCollectionDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Generic List of string type and adding elements using
initializer
            List<string> countries = new List<string>
            {
                "INDIA",
                "USA",
                "JAPAN",
                "UK"
            };
            //Accessing List Elements using ForEach Loop
            Console.WriteLine("Accessing Generic List Elemenst using For Each
foreach (var item in countries)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

A screenshot of a terminal window with a black background and light blue/grey text. The output shows the text "Accessing Generic List Elemenst using For Each Loop" followed by four lines of country names: "INDIA", "USA", "JAPAN", and "UK".

```
Accessing Generic List Elemenst using For Each Loop
INDIA
USA
JAPAN
UK
```

How to Insert Elements at a Specific Position in a Generic List Collection in C#?

If you want to Insert a specific Element or a collection of Elements at a specific position in a Generic List Collection, then you need to use the following two methods provided by the Generic List Collection Class in C#. The Insert method is used to insert a specific element and the InsertRange method is used to insert a collection of elements.

1. **Insert(int index, T item):** This method is used to insert an element into the Generic List at the specified index. Here, the parameter index specifies the zero-based index at which an item should be inserted and the parameter item specifies the object to insert. The value can be null for reference types. If the index is less than 0 or the index is greater than Generic List Count, then it will throw `ArgumentOutOfRangeException`.
2. **InsertRange(int index, IEnumerable<T> collection):** This method is used to insert the elements of a collection into the Generic List at the specified index. Here, the parameter index specifies the zero-based index at which an item should be inserted. The parameter collection specifies the collection whose elements should be inserted into the Generic List. The collection itself cannot be null, but it can contain elements that are null if type T is a reference type. If the collection is null, then it will throw `ArgumentNullException`. If the index is less than 0 or the index is greater than Generic List Count, then it will throw `ArgumentOutOfRangeException`.

For a better understanding of Insert and InsertRange Methods of Generic List Collection Class in C#, please have a look at the below example. The following example code is self-explained, so please go through the comment lines.

```
using System;
using System.Collections.Generic;
namespace GenericListCollectionDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Generic List of string type
            List<string> countries = new List<string>
            {
                "INDIA",
                "USA"
            };
            Console.WriteLine("Accessing List Elements Before Inserting");
            foreach (var item in countries)
            {
                Console.WriteLine(item);
            }
            //Insert Element at Index Position 1
            countries.Insert(1, "China");
            Console.WriteLine($"\\nIndex of China Element in the List :
{countries.IndexOf("China")}");
            //Accessing List After Insert Method
            Console.WriteLine("\\nAccessing List After Inserting China At Index");
            foreach (var item in countries)
            {
                Console.WriteLine(item);
            }
            //Creating Another Generic List Collection of string type
            List<string> newCountries = new List<string>
            {
                "JAPAN",
                "UK"
            };
        }
    }
}
```

```

        //Inserting the newCountries collection into the existing countries
        //Index 2 using InsertRange Method
        countries.InsertRange(2, newCountries);
        //Accessing countries List After InsertRange Method
        Console.WriteLine("\nAccessing List After InsertRange At Index 2");
        foreach (var item in countries)
        {
            Console.WriteLine(item);
        }
        Console.ReadKey();
    }
}

```

Output:

```

Accessing List Elements Before Inserting
INDIA
USA

Index of China Element in the List : 1

Accessing List After Inserting China At Index 1
INDIA
China
USA

Accessing List After InsertRange At Index 2
INDIA
China
JAPAN
UK
USA

```

How to Check the Availability of an Element in a List Collection in C#?

If you want to check whether an element exists or not in the Generic List Collection, then you need to use the following Contains method of the Generic List Collection Class in C#.

1. **Contains(T item):** The Contains(T item) method of the Generic List Collection Class is used to check if the given item is present in the List or not. The parameter item specifies the object to locate in the Generic List. The value can be null for reference types. It returns true if the item is found in the Generic List; otherwise, false.

Let us understand this with an example. The following example shows how to use the Contains method of the Generic List Collection class in C# to check whether an element exists or not in the collection.

```
using System;
```

```

using System.Collections.Generic;
namespace GenericListCollectionDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Generic List of string type and adding elements using
initializer
            List<string> countries = new List<string>
            {
                "INDIA",
                "USA",
                "JAPAN",
                "UK"
            };
            //Accessing List Elements using ForEach Loop
            Console.WriteLine("All Generic List Elementst");
            foreach (var item in countries)
            {
                Console.WriteLine(item);
            }
            //Checking the Item using the Contains method
            Console.WriteLine("\nIs INDIA Exists in List: " +
countries.Contains("INDIA"));
            Console.WriteLine("Is NZ Exists in List: " + countries.Contains("N
Console.ReadKey();
        }
    }
}

```

Output:

```

All Generic List Elementst
INDIA
USA
JAPAN
UK

Is INDIA Exists in List: True
Is NZ Exists in List: False

```

How to Remove Elements from a Generic List Collection in C#?

If you want to remove elements from the list, then you can use the following methods of the List collection class.

1. **Remove(T item):** This method is used to remove the first occurrence of a specific object from the Generic List. Here, the parameter item specifies the object to remove from the Generic List. It returns

true if the item is successfully removed; otherwise, false. This method also returns false if the item was not found in the Generic List.

2. **RemoveAll(Predicate<T> match):** This method is used to remove all the elements that match the conditions defined by the specified predicate. Here, the parameter match specifies the predicate delegate that defines the conditions of the elements to remove. It returns the number of elements removed from the Generic List. If the parameter match is null, then it will throw ArgumentNullException.
3. **RemoveAt(int index):** This method is used to remove the element at the specified index of the Generic List. Here, the parameter index is the zero-based index of the element to remove. If the index is less than 0 or the index is equal to or greater than Generic List Count, then it will throw ArgumentOutOfRangeException.
4. **RemoveRange(int index, int count):** This method is used to remove a range of elements from the Generic List. Here, the parameter index is the zero-based starting index of the range of elements to remove and the parameter count is the number of elements to remove. If the index is less than 0 or the count is less than 0, then it will throw ArgumentOutOfRangeException. If the index and count do not denote a valid range of elements in the Generic List, then it will throw ArgumentException.
5. **Clear():** This method is used to remove all elements from the Generic List.

For a better understanding of how to use the above methods of the Generic List Collection Class in C#, please have a look at the below example.

```
using System;
using System.Collections.Generic;
namespace GenericListCollectionDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Generic List of string type and adding elements using
initializer
            List<string> countries = new List<string>
            {
                "INDIA",
                "USA",
                "JAPAN",
                "UK",
                "AUSTRALIA",
                "SRILANKA",
                "BANGLADESG",
                "NEPAL",
                "CHINA",
                "NZ",
                "SOUTH AFRICA"
            };
            Console.WriteLine($"Before Removing Element Count : {countries.Count}");
            //Using Remove method to Remove an Element from the List
            Console.WriteLine($"Removing Element SRILANKA :
{countries.Remove("SRILANKA")}");
            Console.WriteLine($"After Removing SRILANKA Element Count :
{countries.Count}");
            //Removing Element using Index Position from the List
```

```

        countries.RemoveAt(2);
        Console.WriteLine($"\\nAfter Removing Index 2 Element Count :
{countries.Count}");
        // Using RemoveAll method to Remove Elements from the List
        // Here, we are removing element whose length is less than 3 i.e.
        //countries.RemoveAll(x => x.Length < 3);
        Console.WriteLine($"\\nRemoveAll Method Removes: {countries.Remove
x.Length < 3)} Element(s)");
        Console.WriteLine($"After RemoveAll Method Element Count :
{countries.Count}");
        //Removing Element using RemoveRange(int index, int count) Method
        //Here, we are removing the first two elements
        countries.RemoveRange(0, 2);
        Console.WriteLine($"\\nAfter RemoveRange Method Element Count :
{countries.Count}");
        //Removing All Elements using Clear method
        countries.Clear();
        Console.WriteLine($"\\nAfter Clear Method Element Count : {countri
        Console.ReadKey();
    }
}
}

```

Output:

```

Before Removing Element Count : 11
Removing Element SRILANKA : True
After Removing SRILANKA Element Count : 10
After Removing Index 2 Element Count : 9
RemoveAll Method Removes: 2 Element(s)
After RemoveAll Method Element Count : 7
After RemoveRange Method Element Count : 5
After Clear Method Element Count : 0

```

How to Copy an Array to a List in C#?

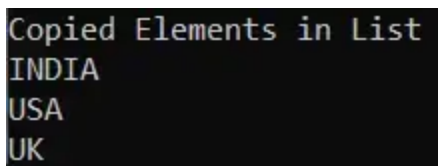
To Copy an Array to a List we need to use the following overloaded constructor of the List class in C#. As you

1. **public List(IEnumerable<T> collection):** This constructor is used to initialize a new instance of the Generic List class that contains elements copied from the specified collection and has sufficient capacity to accommodate the number of elements copied. The parameter collection specifies the collection whose elements are copied to the new list.

For a better understanding please have a look at the below example. Here we create a List with elements from an array. We use the List constructor and pass the array as an argument. The list receives this parameter and fills its values from it. The array element type must match the List element type or the compilation will fail.

```
using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            // Create new array with 3 elements.
            string[] array = new string[] { "INDIA", "USA", "UK" };
            // Copy the array to a List.
            List<string> copiedList = new List<string>(array);
            Console.WriteLine("Copied Elements in List");
            foreach (var item in copiedList)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:



```
Copied Elements in List
INDIA
USA
UK
```

As of now, we are working with Generic List Collection with built-in primitive data types. Now, let us proceed further and try to understand how to work with Complex Data Types like Student, Employee, Product, etc.

Generic List Collection with Complex Type in C#:

Let us see an example of the List Collection class with Complex Types in C#. Please have a look at the below example code. As you can see in the below code, we have created a class called Employee. Then we create a few employee objects and then we create a Generic List Collection of type Employee and store all the employee objects in that collection. Finally, we are performing different types of operations using the methods provided by the List<T> Generic Collection Class. The following example code is self-explained. So please go through the comment lines.

```
using System;
using System.Collections.Generic;
namespace GenericListCollectionDemo
{
```

```

public class Program
{
    public static void Main()
    {
        // Create Employee Objects
        Employee emp1 = new Employee() { ID = 101, Name = "Pranaya", Gender = "Male", Salary = 5000 };
        Employee emp2 = new Employee() { ID = 102, Name = "Priyanka", Gender = "Female", Salary = 7000 };
        Employee emp3 = new Employee() { ID = 103, Name = "Anurag", Gender = "Male", Salary = 5500 };

        // Create a Generic List of type Employee
        List<Employee> listEmployees = new List<Employee>();
        //Adding Employees to the listEmployees collection using Add Method
        listEmployees.Add(emp1);
        listEmployees.Add(emp2);
        listEmployees.Add(emp3);
        Console.WriteLine("Retrive the First Employee By Index");
        //Retrieving Items from the collection by using Index.
        //The following line of code will retrieve the employee from the list
        //The List index is 0 based.
        Employee FirstEmployee = listEmployees[0]; //Fetch the First Added Employee
        Console.WriteLine($"ID = {FirstEmployee.ID}, Name = {FirstEmployee.Name}, Gender = {FirstEmployee.Gender}, Salary = {FirstEmployee.Salary}");
        //Retrieving All Employees of Generic List Collection using For Loop
        Console.WriteLine("\nRetrieving All Employees using For Loop");
        for (int i = 0; i < listEmployees.Count; i++)
        {
            Employee employee = listEmployees[i];
            Console.WriteLine($"ID = {employee.ID}, Name = {employee.Name}, Gender = {employee.Gender}, Salary = {employee.Salary}");
        }
        //Retrieving All Employees of Generic List Collection using For Each Loop
        Console.WriteLine("\nRetrieving All Employees using For Each Loop");
        foreach (Employee employee in listEmployees)
        {
            Console.WriteLine($"ID = {employee.ID}, Name = {employee.Name}, Gender = {employee.Gender}, Salary = {employee.Salary}");
        }
        //Inserting an Employee into the Index Position 1.
        Employee emp4 = new Employee() { ID = 104, Name = "Sambit", Gender = "Male", Salary = 6500 };
        listEmployees.Insert(1, emp4);
        //Retrieving the list after inserting the employee in index position 1
        Console.WriteLine("\nRetriving the List After Inserting New Employee at Index Position 1");
        foreach (Employee employee in listEmployees)
        {

```

```

        Console.WriteLine($"ID = {employee.ID}, Name = {employee.Name}
{employee.Gender}, Salary = {employee.Salary}");
    }
    //If you want to get the index position of a specific employee then
    IndexOf() method as follows
    Console.WriteLine("\nIndex of emp3 object in the List = " +
listEmployees.IndexOf(emp3));
    Console.ReadKey();
}
}
}
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}
}
}

```

Output:

```

Retrive the First Employee By Index
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000

Retrieving All Employees using For Loop
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 103, Name = Anurag, Gender = Male, Salary = 5500

Retrieving All Employees using For Each Loop
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 103, Name = Anurag, Gender = Male, Salary = 5500

Retriving the List After Inserting New Employee in Index 1
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 104, Name = Sambit, Gender = Male, Salary = 6500
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 103, Name = Anurag, Gender = Male, Salary = 5500

Index of emp3 object in the List = 3

```

Note: All the Generic Collection Classes in C# are Strongly Typed. That means if we have created a List of type Employee, then we can only add objects of type Employee into the list. If we try to add an object of a different type, then we will get a compile-time error.

How to Find Element in a Generic List Collection in C#?

The Generic List Collection Class in C# provides a lot of useful methods that we can use to find elements on a collection of List Types. The List Collection class provides the following important methods to find elements in a collection.

1. **Find():** The Find() method is used to find the first element from a list based on a condition that is specified by a lambda expression.
2. **FindLast():** The FindLast() method is used to search for an element that matches the conditions specified by a predicate. If it found any elements with that specified condition then it returns the Last matching element from the list.
3. **FindAll():** The FindAll() method is used to retrieve all the elements from a list that matches the conditions specified by a predicate.
4. **FindIndex():** The FindIndex() method is used to return the index position of the first element that matches the conditions specified by a predicate. The point that you need to remember is the index here in generic collections is zero-based. This method returns -1 if an element that matches the specified conditions is not found. There are 2 other overloaded versions of this method is available, one of the overload versions allows us to specify the range of elements to search within the list.
5. **FindLastIndex():** The FindLastIndex() Method searches for an element in the list that matches the condition specified by the lambda expression and then returns the index of the last occurrence of the item within the list. There are 2 other overloaded versions of this method is available, one of the overload versions allows us to specify the range of elements to search within the list.
6. **Exists():** The Exists() method is used to check or determine whether an item exists or not in a list based on a condition. If the item exists then it will return true else it will return false.
7. **Contains():** The Contains() method is used to determine whether the specified item exists or not in the list. If the specified item exists then it will return true else return false.

Let us understand all the above methods of the List Collection class in C# with an Example. The following example is self-explained, so please go through the comment lines.

```
using System;
using System.Collections.Generic;
namespace ListCollectionDemo
{
    public class Program
    {
        public static void Main()
        {
            // Create Employee Objects
            Employee Employee1 = new Employee() { ID = 101, Name = "Pranaya",
"Male", Salary = 5000 };
            Employee Employee2 = new Employee() { ID = 102, Name = "Priyanka",
"Female", Salary = 7000 };
            Employee Employee3 = new Employee() { ID = 103, Name = "Anurag", (
"Male", Salary = 5500 };
            Employee Employee4 = new Employee() { ID = 104, Name = "Sambit", (
"Male", Salary = 6500 };
            //Creating a list of type Employee
            List<Employee> listEmployees = new List<Employee>
            {
                Employee1,
                Employee2,
                Employee3,
                Employee4
            }
        }
    }
}
```

```

};
// use Contains method to check if an item exists or not in the list
Console.WriteLine("Contains Method Check Employee2 Object");
if (listEmployees.Contains(Employee2))
{
    Console.WriteLine("Employee2 Object Exists in the List");
}
else
{
    Console.WriteLine("Employee2 Object Does Not Exist in the List");
}
// Use Exists method when you want to check if an item exists or not in the list based on a condition
Console.WriteLine("\nExists Method Name Starts With P");
if (listEmployees.Exists(x => x.Name.StartsWith("P")))
{
    Console.WriteLine("List contains Employees whose Name Starts With P");
}
else
{
    Console.WriteLine("List does not Contain Any Employee whose Name Starts With P");
}
// Use Find() method, if you want to return the First Matching Element based on conditions
Console.WriteLine("\nFind Method to Return First Matching Employee whose Gender = Male");
Employee? emp = listEmployees.Find(employee => employee.Gender == "Male");
Console.WriteLine($"ID = {emp?.ID}, Name = {emp?.Name}, Gender = {emp?.Gender}, Salary = {emp?.Salary}");
// Use FindLast() method when you want to search an item by a condition and it returns the Last matching item from the list
Console.WriteLine("\nFindLast Method to Return Last Matching Employee whose Gender = Male");
Employee? lastMatchEmp = listEmployees.FindLast(employee => employee.Gender == "Male");
Console.WriteLine($"ID = {lastMatchEmp?.ID}, Name = {lastMatchEmp?.Name}, Gender = {lastMatchEmp?.Gender}, Salary = {lastMatchEmp?.Salary}");
// Use FindAll() method when you want to return all the items that match the conditions
Console.WriteLine("\nFindAll Method to return All Matching Employees whose Gender = Male");
List<Employee> filteredEmployees = listEmployees.FindAll(employee => employee.Gender == "Male");
foreach (Employee femp in filteredEmployees)
{
    Console.WriteLine($"ID = {femp.ID}, Name = {femp.Name}, Gender = {femp.Gender}, Salary = {femp.Salary}");
}

```

```

        // Use FindIndex() method when you want to return the index of the
        by a condition
        Console.WriteLine($"Index of the First Matching Employee whose Gender = Male = {listEmployees.FindIndex(employee => employee.Gender == "Male")}");

        // Use FindLastIndex() method when you want to return the index of
        item by a condition
        Console.WriteLine($"Index of the Last Matching Employee whose Gender = Male = {listEmployees.FindLastIndex(employee => employee.Gender == "Male")}");
        Console.ReadKey();
    }
}

public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}
}

```

Output:

```

Contains Method Check Employee2 Object
Employee2 Object Exists in the List

Exists Method Name StartsWith P
List contains Employees whose Name Starts With P

Find Method to Return First Matching Employee whose Gender = Male
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000

FindLast Method to Return Last Matching Employee whose Gender = Male
ID = 104, Name = Sambit, Gender = Male, Salary = 6500

FindAll Method to return All Matching Employees Where Gender = Male
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 104, Name = Sambit, Gender = Male, Salary = 6500

Index of the First Matching Employee whose Gender is Male = 0
Index of the Last Matching Employee whose Gender is Male = 3

```

Generic List Class important methods in C#:

1. **TrueForAll():** This method returns true or false depending on whether every element in the list matches the conditions defined by the specified predicate.

2. **AsReadOnly():** This method returns a read-only wrapper for the current collection. Use this method, if you don't want the client to modify the collection i.e. add or remove any elements from the collection. The ReadOnlyCollection will not have methods to add or remove items from the collection. We can only read items from this collection.
3. **TrimExcess():** This method sets the capacity to the actual number of elements in the List if that number is less than a threshold value.

Example to Understand the above three methods of Generic List Collection Class in C#.

```
using System;
using System.Collections.Generic;
namespace ListCollectionDemo
{
    public class Program
    {
        public static void Main()
        {
            //Creating a list of type Employee
            List<Employee> listEmployees = new List<Employee>
            {
                new Employee() { ID = 101, Name = "Pranaya", Gender = "Male", Salary = 5000 },
                new Employee() { ID = 102, Name = "Priyanka", Gender = "Female", Salary = 7000 },
                new Employee() { ID = 103, Name = "Anurag", Gender = "Male", Salary = 5500 },
                new Employee() { ID = 104, Name = "Sambit", Gender = "Male", Salary = 6500 },
                new Employee() { ID = 105, Name = "Hina", Gender = "Female", Salary = 6500 }
            };
            //TrueForAll
            Console.WriteLine($"Are all salaries greater than 5000: {listEmployees.TrueForAll(x => x.Salary > 5000)}");

            // ReadOnlyCollection will not have Add() or Remove() methods
            System.Collections.ObjectModel.ReadOnlyCollection<Employee> readOnlyEmployees = listEmployees.AsReadOnly();
            Console.WriteLine($"Total Items in ReadOnlyCollection: {readOnlyEmployees.Count}");
            // listEmployees list is created with an initial capacity of 8
            // but only 5 items are in the list. The filled percentage is less than the default
            // percent threshold.
            Console.WriteLine($"List Capacity Before invoking TrimExcess: {listEmployees.Capacity}");
            // Invoke TrimExcess() to set the capacity to the actual number of elements
            // in the List
            listEmployees.TrimExcess();
        }
    }
}
```

```

        Console.WriteLine($"\\nList Capacity After invoking TrimExcess:
{listEmployees.Capacity} ");
        Console.ReadKey();
    }
}
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}
}

```

Output:

```

Are all salaries greater than 5000: False
Total Items in ReadOnlyCollection: 5
List Capacity Before invoking TrimExcess: 8
List Capacity After invoking TrimExcess: 5

```

How to Sort a List of Simple Types in C#?

In C#, sorting a list of simple types like int, double, char, string, etc. is straightforward. Here, we just need to call the Sort() method which is provided by the Generic List class on the list instance, and then the data will be automatically sorted in ascending order. For example, if we have a list of integers as shown below.

```
List<int> numbersList = new List<int>{ 1, 8, 7, 5, 2};
```

Then we just need to invoke the Sort() method on numbersList collection as shown below

```
numbersList.Sort();
```

If you want the data to be retrieved in descending order, then use the Reverse() method on the list instance as shown below.

```
numbersList.Reverse();
```

Example to Understand Generic List Collection Class Sort and Reverse Method in C#:

```

using System;
using System.Collections.Generic;
namespace ListCollectionSortReverseMethodDemo
{
    public class Program
    {
        public static void Main()

```

```

{
    List<int> numbersList = new List<int> { 1, 8, 7, 5, 2 };
    Console.WriteLine("Numbers Before Sorting");
    foreach (int i in numbersList)
    {
        Console.Write($"{i} ");
    }
    // The Sort() of List Collection class will sort the data in ascer
    numbersList.Sort();
    Console.WriteLine("\n\nNumbers After Sorting");
    foreach (int i in numbersList)
    {
        Console.Write($"{i} ");
    }
    // If you want to to retrieve data in descending order then use 1
Reverse() method
    numbersList.Reverse();
    Console.WriteLine("\n\nNumbers in Descending order");
    foreach (int i in numbersList)
    {
        Console.Write($"{i} ");
    }
    //Another Example of Sorting String
    List<string> names = new List<string>() { "Pranaya", "Anurag", "S:
"Hina", "Rakesh"};
    Console.WriteLine("\n\nNames Before Sorting");
    foreach (string name in names)
    {
        Console.WriteLine(name);
    }
    names.Sort();
    Console.WriteLine("\nNames After Sorting");
    foreach (string name in names)
    {
        Console.WriteLine(name);
    }
    names.Reverse();
    Console.WriteLine("\nNames in Descending Order");
    foreach (string name in names)
    {
        Console.WriteLine(name);
    }
    Console.ReadKey();
}
}
}

```

Output:

```
Numbers Before Sorting
1 8 7 5 2

Numbers After Sorting
1 2 5 7 8

Numbers in Descending order
8 7 5 2 1

Names Before Sorting
Pranaya
Anurag
Sambit
Hina
Rakesh

Names After Sorting
Anurag
Hina
Pranaya
Rakesh
Sambit

Names in Descending Order
Sambit
Rakesh
Pranaya
Hina
Anurag
```

However, when we do the same thing on a complex type like Employee, Product, Customer, Department, etc. we will get a runtime exception as “**invalid operation exception – Failed to compare 2 elements in the array**”. For a better understanding, please have a look at the below example. Here, we are creating a collection of Employees and then calling the Sort method which will throw runtime exception.

```
using System;
using System.Collections.Generic;
namespace ListCollectionDemo
{
    public class Program
    {
        public static void Main()
        {
            //Creating a list of type Employee
            List<Employee> listEmployees = new List<Employee>
            {
                new Employee() { ID = 101, Name = "Pranaya", Gender = "Male",
5000 },
```

```

        new Employee() { ID = 102, Name = "Priyanka", Gender = "Female", Salary = 7000 },
        new Employee() { ID = 103, Name = "Anurag", Gender = "Male", Salary = 5500 },
        new Employee() { ID = 104, Name = "Sambit", Gender = "Male", Salary = 6500 },
        new Employee() { ID = 105, Name = "Hina", Gender = "Female", Salary = 6500 }
    };
    listEmployees.Sort();
    Console.ReadKey();
}
}
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}
}

```

Output:

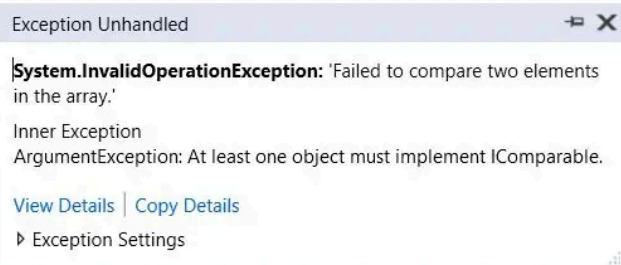
```

public class Program
{
    public static void Main()
    {
        //Creating a list of type Employee
        List<Employee> listEmployees = new List<Employee>
        {
            new Employee() { ID = 101, Name = "Pranaya", Gender = "Male", Salary = 5000 },
            new Employee() { ID = 102, Name = "Priyanka", Gender = "Female", Salary = 7000 },
            new Employee() { ID = 103, Name = "Anurag", Gender = "Male", Salary = 5500 },
            new Employee() { ID = 104, Name = "Sambit", Gender = "Male", Salary = 6500 },
            new Employee() { ID = 105, Name = "Hina", Gender = "Female", Salary = 6500 }
        };

        listEmployees.Sort();

        Console.ReadKey();
    }
}
public class Employee

```



We are getting the above Runtime Exception because at runtime the .NET Framework does not identify how to sort the complex types. In this, the .NET Runtime is unable to identify whether to sort the data based on ID property, or based on Name property, or based on Gender property, or based on Salary property, or a combination of any properties. So, if we want to sort a complex type then we need to tell the way we want the data to be sorted in the list, and to do this we need to implement the IComparable interface. We will discuss this in our next article.

How the sort functionality is working for simple data types like int, double, string, char, etc. in C#?

This is working because these types (int, double, string, decimal, char, etc.) are already implementing the `IComparable` interface. If you go to the definition of any built-in types, then you will see that the class is implemented `IComparable` interface as shown in the below image and this is the reason why the sort functionality is working.

```
...public struct Int32 : IComparable, IFormattable, IConvertible, IComparable<Int32>, IEquatable<Int32>
{
    ...public const Int32 MaxValue = 2147483647;
    ...public const Int32 MinValue = -2147483648;

    ...public sealed class String : IComparable, ICloneable, IConvertible, IEnumerable,
    {
        ...public static readonly String Empty;
```

Summary of Generic List<T> Collection Class in C#:

1. The List<T> collection is different from the arrays. The List can be resized dynamically but arrays cannot resize dynamically.
2. The Generic List<T> Collection Class in C# can accept null values for reference types and it also accepts duplicate values.
3. When the element's Count becomes equal to the Capacity of the list collection, then the capacity of the List increased automatically by reallocating the internal array. The existing elements will be copied to the new array before the addition of the new element.
4. The Generic List class is the generic equivalent of the Non-Generic ArrayList class.
5. The Generic List<T> class implements the IList<T> generic interface.
6. We can use both equality and ordering comparer with the generic List class.
7. The List<T> class elements are not sorted by default and elements are accessed by a zero-based index.
8. For very large List<T> objects, you can increase the maximum capacity to 2 billion elements on a 64-bit system by setting the enabled attribute of the configuration element to true in the run-time environment.

In the next article, I am going to discuss [How to Sort a List of Complex Types in C#](#) with Examples. Here, in this article, I try to explain the **Generic List<T> Collection Class in C#** with Examples. I hope this Generic List Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.